

UML: DIAGRAMA DE CLASSES

Parte do material foi adaptado das aulas da PUC-RIO

UML — DIAGRAMA DE CLASSES

Revisão de Conceitos

- Conceitos OO aplicados em classes

Introdução

Elementos do diagrama de classes

Revisão de Conceitos

CONCEITOS— O QUE É CLASSE?

Metáfora da **Engenharia Civil**:

Lembra da **casa** que **queríamos construir** algum tempo atrás.

Pra começar **a construir** fizemos **todo o planejamento** e seguimos **um processo**, passamos **pelo Arquiteto, engenheiro** que elaboraram todas as **plantas da casa**.

Agora vamos para **uma nova construção**:

CONCEITOS — O QUE É CLASSE?

- Agora **temos** a incumbência **de construir um prédio** com **vários apartamentos**.
- Nesse **prédio** todos **os apartamentos** serão **rigorosamente iguais**.
- **Baseados em uma planta** que anteriormente já esclarecia como seria cada apartamento:
 - Os números de dormitórios
 - Postais
 - Sanitários
 - Fiação elétrica, e
 - Disposição hidráulica, etc.

CONCEITOS — O QUE É CLASSE?

- A planta dessa casa = Classe
 - um conjunto de objetos com características semelhantes
- Os Apartamentos que compõe Classe = Objetos
- As Os cômodos desses Objetos = Atributos

CONCEITOS — INSTÂNCIA DE UMA CLASSE

- A criação de um Objeto baseado em uma classe recebe um nome especial.
- Ele é chamado de **INSTÂNCIA**.
 - um objeto cujo comportamento e estado são definidos pela classe.

Objeto 1



Objeto 2



Classe Automovel
numeroPortas cor fabricante ano placa

CONCEITOS — CLASSES ABSTRATAS E CONCRETAS

Abstratas são as que **NÃO** permitem realizar qualquer tipo de **instância**.

São classes **feitas especialmente para serem modelos** para suas classes derivadas (Classes Concretas).

CONCEITOS — CLASSES

- **Por exemplo:**
- Um “Veículo” é uma classe ou um molde;
- Vocês já viram, um veículo?

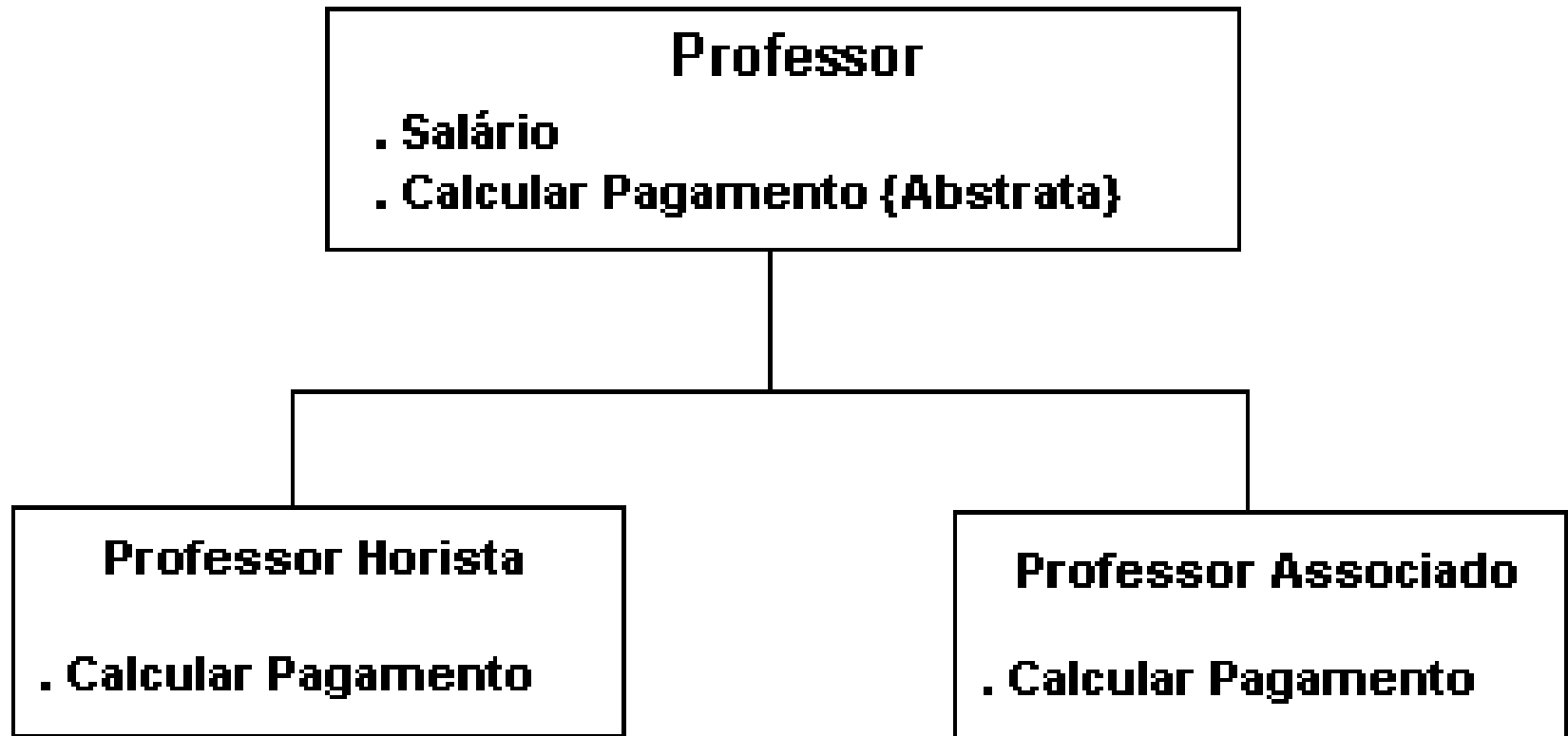
CONCEITOS — CLASSES

- **Por exemplo:**
- Um “Veículo” é uma classe ou um molde;
- Vocês já viram, um veículo?
- Algo que fosse puramente veículo, que não fosse carro, caminhão, ônibus, etc?

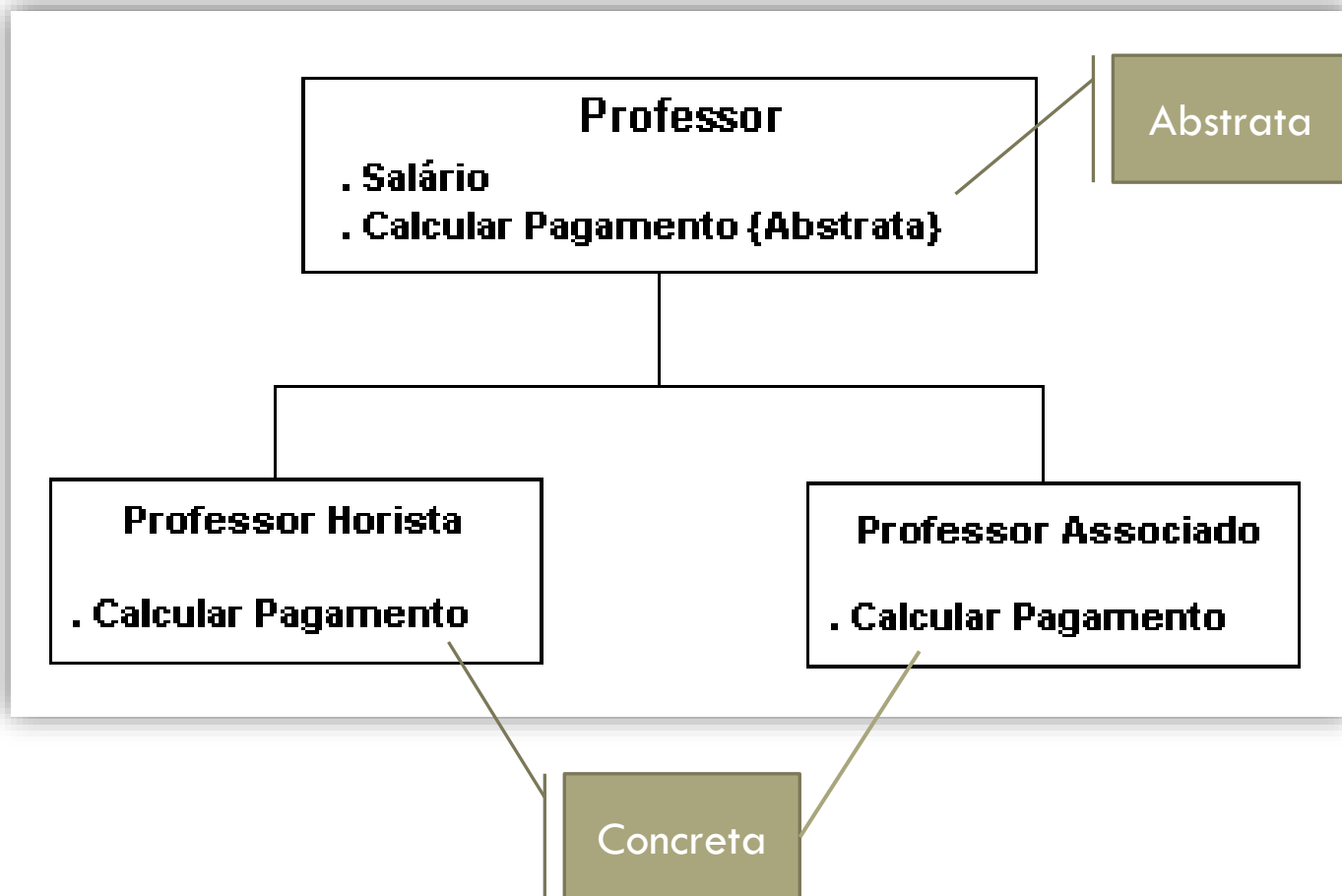
CONCEITOS — CLASSES

- **Por exemplo:**
- Um "animal" é uma classe ou um molde;
- Vocês já viram, um animal?
- Algo que fosse puramente animal, que não fosse humano, cachorro, baleia, cavalo, etc?
- Um "cachorro" é uma Classe concreta de "animal" e a Raça de um cachorro poderia ser a instancia da classe.

CONCEITOS — CLASSES ABSTRATAS



CONCEITOS — CLASSES ABSTRATAS E CONCRETA



CONCEITOS — DIFERENÇA DE OBJETO X CLASSE

- Vimos que um **objeto é uma instancia** de uma classe.
- Talvez essa explicação seja complicada de entender, então vamos melhorar:

CONCEITOS — DIFERENÇA DE OBJETO X CLASSE

- Um **CLASSE**:
 - é qualquer coisa que não tenha seus limites definidos.
 - Ela não existe, se não em uma folha de papel.



Ilustração Artística da Planta do Apartamento Garden de 100m²



CONCEITOS — DIFERENÇA DE OBJETO X CLASSE

- Um **Objeto**:
 - é qualquer coisa que tenha seus limites definidos.



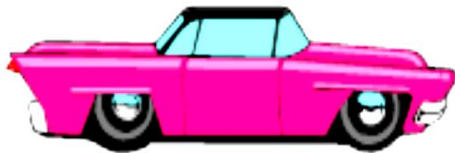
- São OBJETOS

CONCEITOS — DIFERENÇA DE OBJETO X CLASSE

Objeto 1



Objeto 2



Classe
Automovel

numeroPortas
cor
fabricante
ano
placa

CONCEITOS — DIFERENÇA DE OBJETO X CLASSE



MAS PORQUE CHAMAR OBJETO

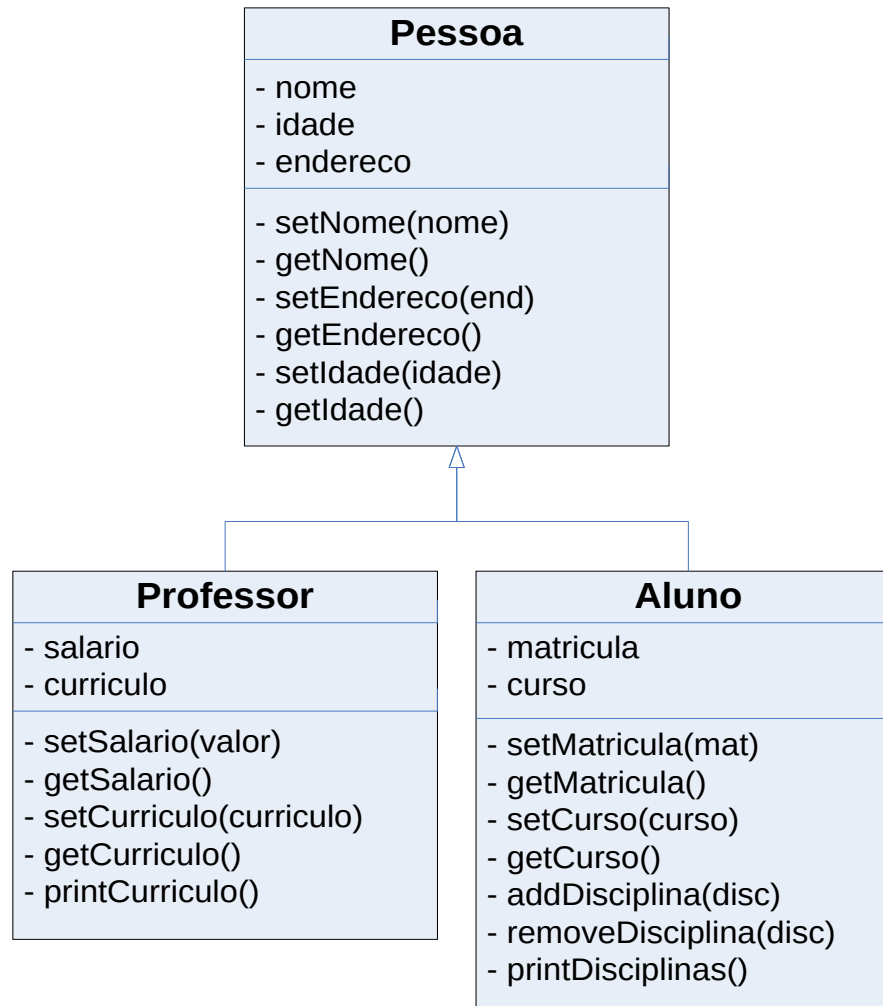
- Na programação:
- Uma vez pensado na altura, largura, cor, formato de uma porta todos os apartamentos são padronizados não justifica pensar em tudo novamente.
- Um **objeto** é construído para **ser reutilizado**.
 - Busca baseada em histórico de conhecimento.

Conceitos de OO aplicados em Classe

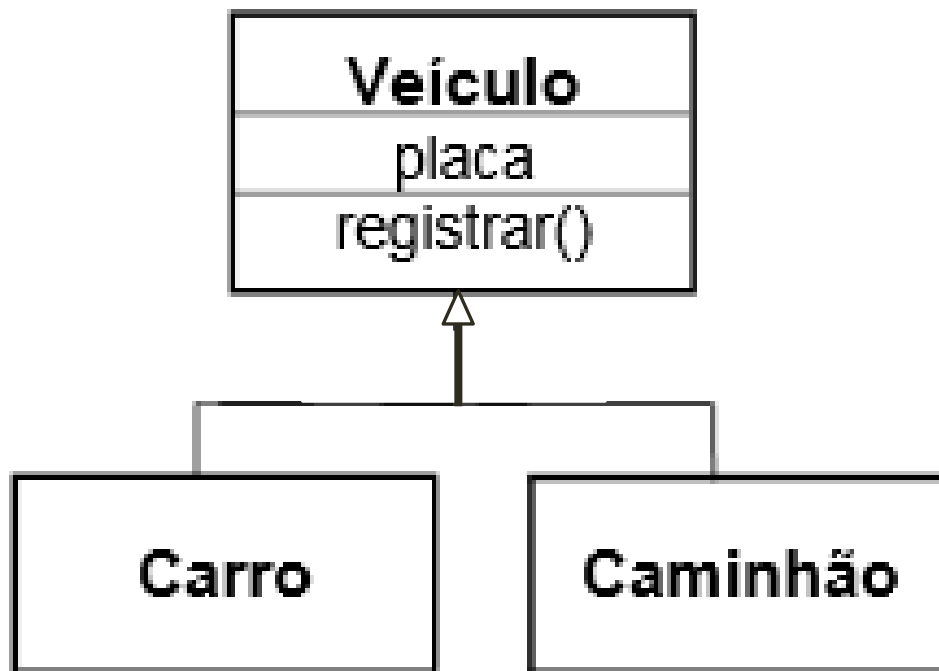
HERANÇA

- **Resumindo:** “Filho de peixe, peixe é”.
- Permite definir **novas classes** (subclasses) a partir de uma classe já existente (superclasse).
- A subclasse herda as **propriedades comuns** da superclasse e pode ainda **adicionar novos métodos** ou **reescrever métodos herdados**.
- **Objetivo:**
 - evitar que classes que possuam **atributos** ou **métodos semelhantes** sejam **repetidamente criados**.

HERANÇA SIMPLES

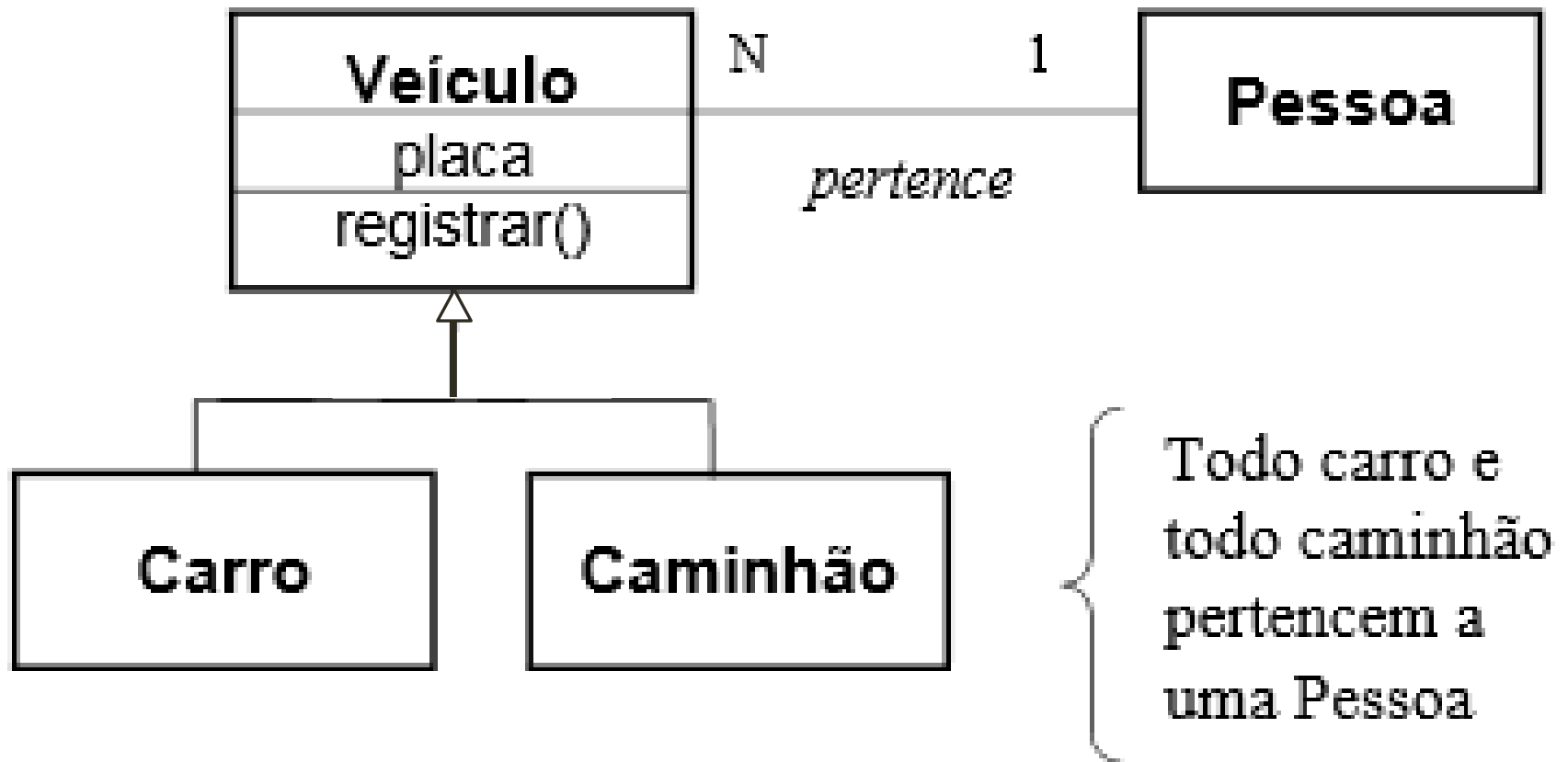


HERANÇA SIMPLES

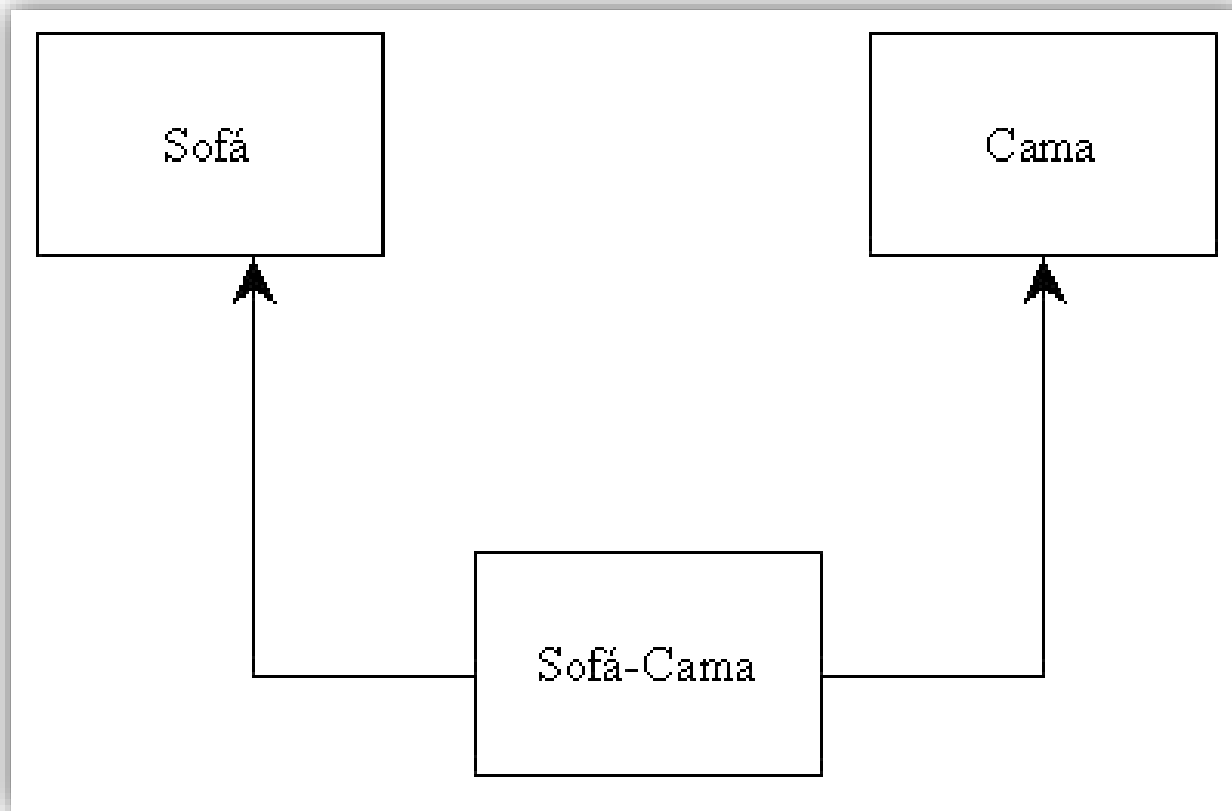


Todo carro
e todo
caminhão
possuem
uma placa
e uma
operação
`registrar()`

HERANÇA DE RELACIONAMENTO



HERANÇA MULTIPLA



HERANÇA MULTIPLA

Relógio



Rádio



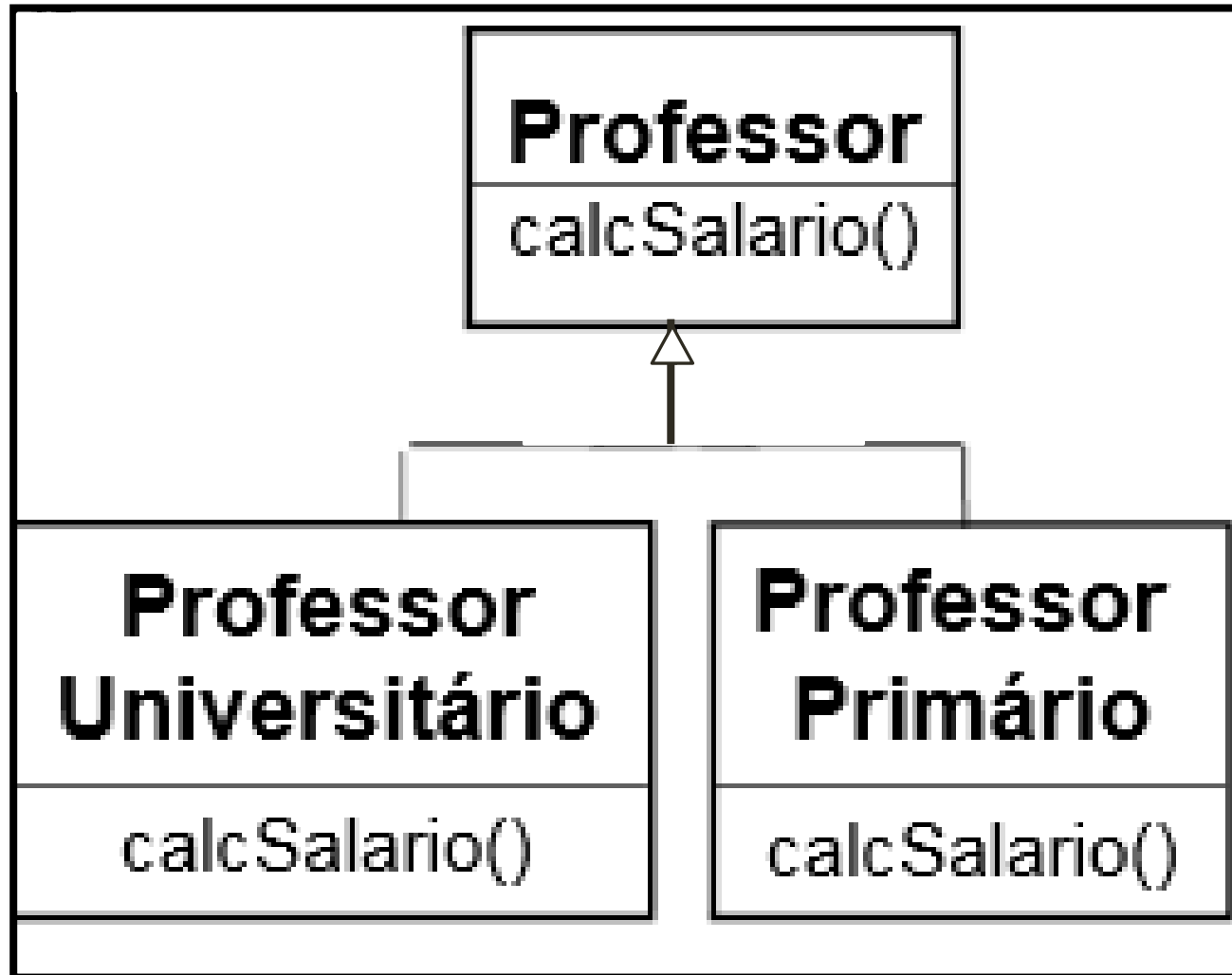
Rádio-Relógio



POLIMORFISMO

- Permite a utilização da mesma operação entre classes genéricas e específicas, porém com implementações (métodos) diferentes.
- Métodos diferentes usando a mesma operação

POLIMORFISMO

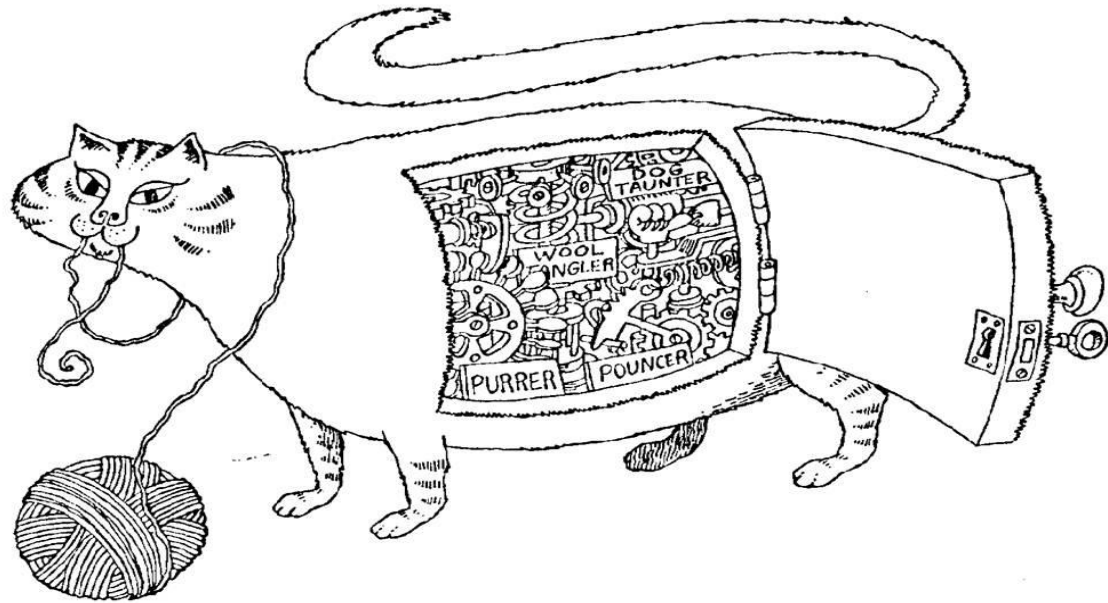


ENCAPSULAMENTO

- Oculta do mundo externo ao objeto os detalhes de implementação e restringir o acesso às propriedades e aos métodos;

ENCAPSULAMENTO

Concepts



Encapsulation hides the details of the implementation of an object.

ACOPLAMENTO

- Forte Dependência entre componentes.
- Quando um módulo depende de outro pra funcionar.
- É difícil adicionar mais funcionalidades sem ter que modificar o código.
- Quanto maior a dependência + ACOMPLADA

COESÃO

- Ligado ao princípio da Responsabilidade Única
- Onde uma classe deve ter uma Única Responsabilidade e realizá-la de maneira Satisfatória.
- Caso ela executa mais de uma coisa, passa a ter dificuldade de manutenção e reuso.

Introdução

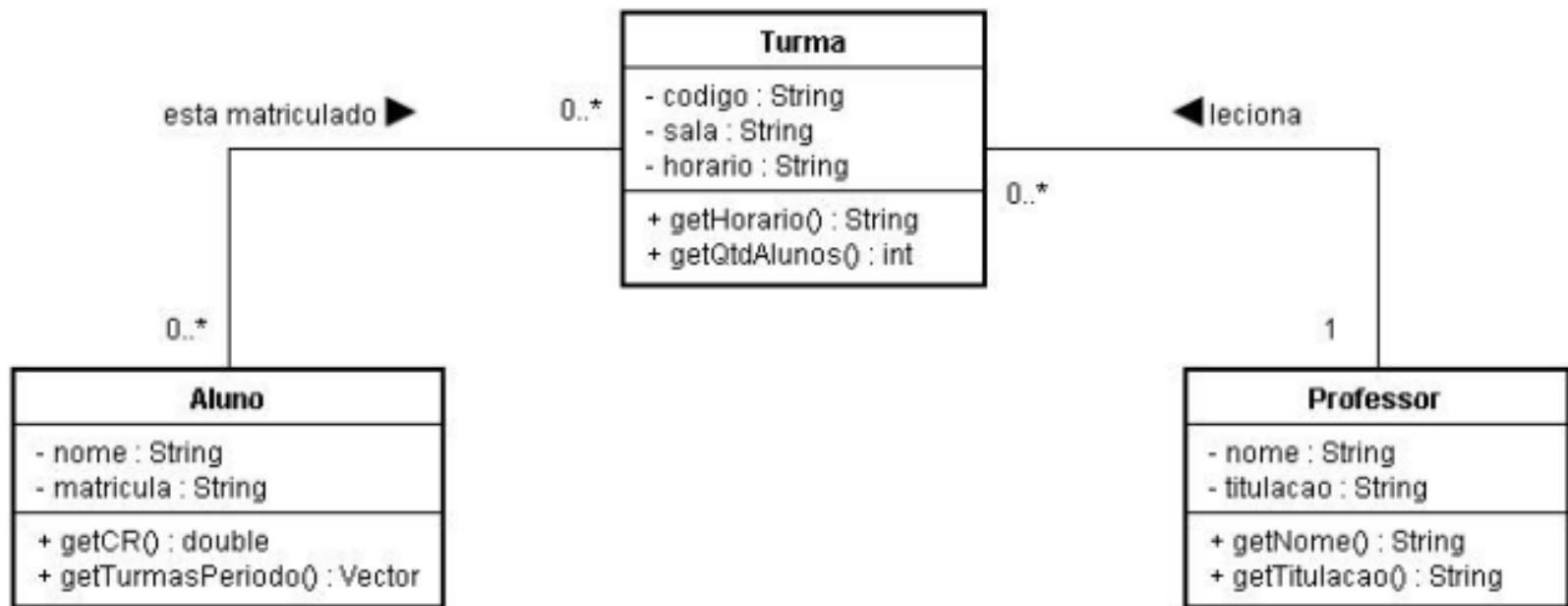
INTRODUÇÃO - DIAGRAMA DE CLASSES

Descreve a **visão estática** de um **sistema** através das classes e dos relacionamentos

É composto por **um ou mais diagramas** de classes

É o **diagrama central da modelagem** orientada a objetos.

INTRODUÇÃO - DIAGRAMA DE CLASSES



Elementos

ELEMENTOS — DIAGRAMA DE CLASSES

Elementos de um diagrama de classes

- Classes
- Relacionamentos
 - Associação
 - Agregação
 - Composição
 - Generalização
 - Dependência

ELEMENTOS — DIAGRAMA DE CLASSES

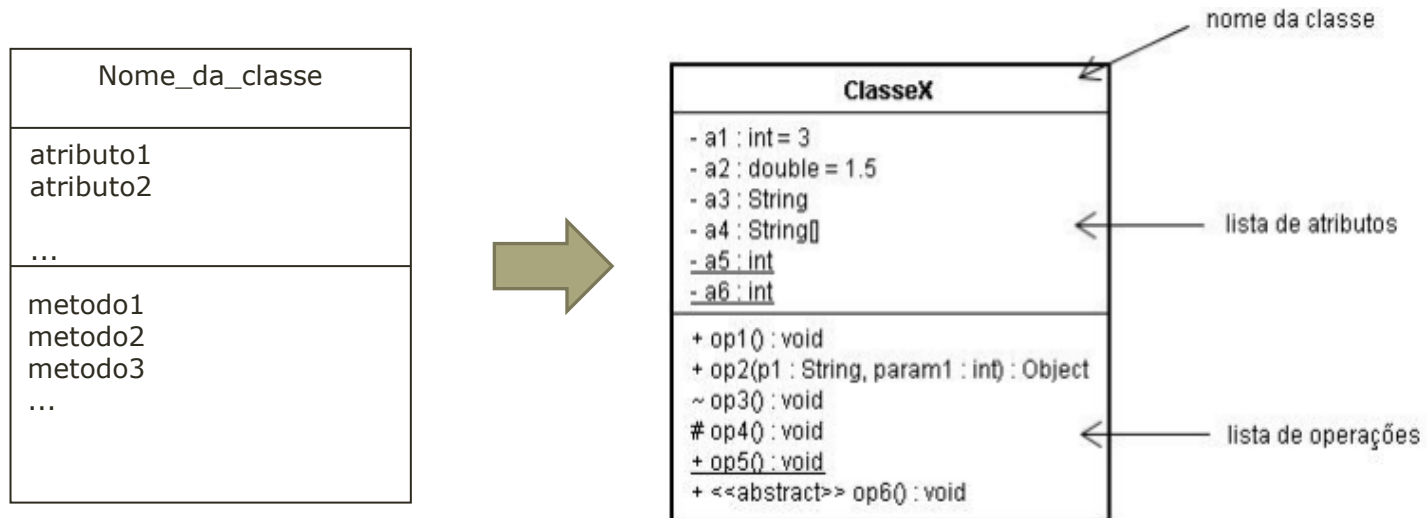
Elementos de um diagrama de classes

- **Classes**
- Relacionamentos
 - Associação
 - Agregação
 - Composição
 - Generalização
 - Dependência

ELEMENTOS — DIAGRAMA DE CLASSES

Classes

Graficamente, **as classes são representadas por retângulos**, dividido em três seções: nome, lista de atributos e lista de métodos ou operações.



ELEMENTOS — DIAGRAMA DE CLASSES

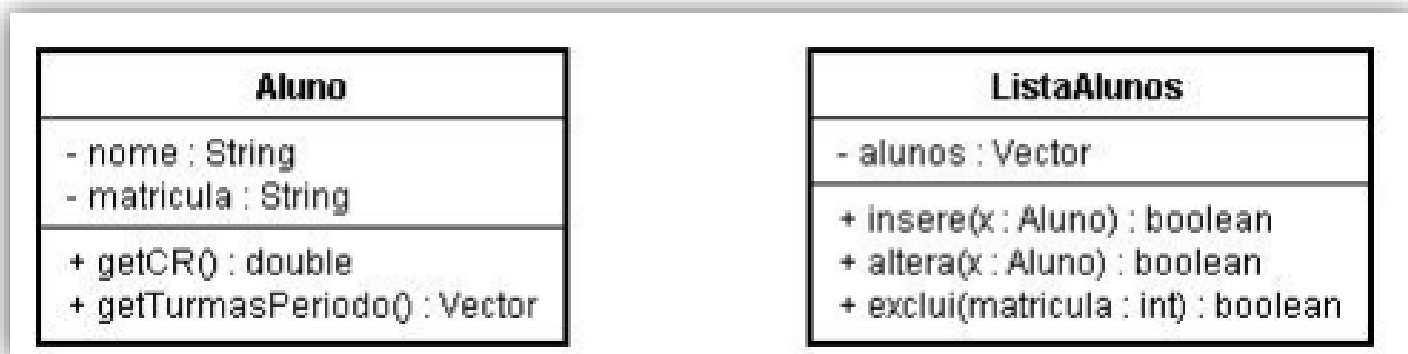
Classes

A UML não define nenhuma regra para a semântica do nome de uma classe

O **nome** deve **ser escolhido** de acordo **com o vocabulário do domínio do problema** ou com **a função exercida** pela classe

É comum adotar um padrão para nomeá-las

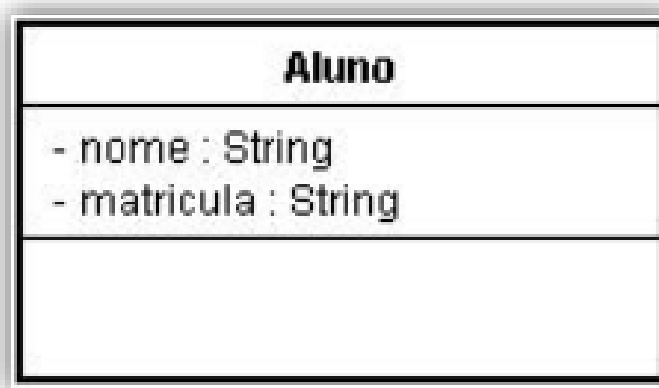
Ex: todos os nomes de classes serão **substantivos singulares** com a **primeira letra maiúscula**



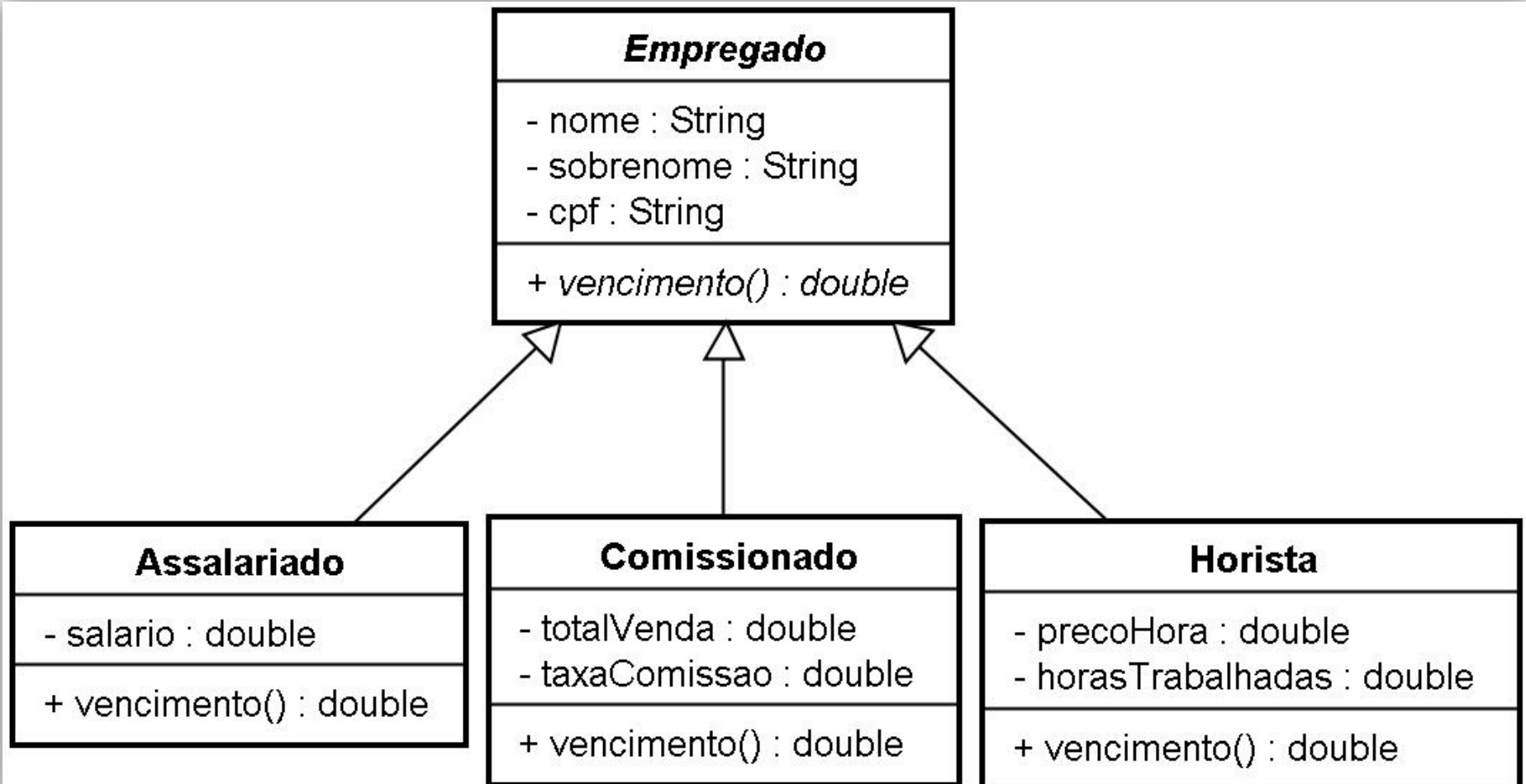
ELEMENTOS — DIAGRAMA DE CLASSES

Atributos

- É uma **propriedade** nomeada de uma **classe** que define um **domínio** de **valores** que um objeto — instância da classe — **pode armazenar**
 - Representam o **conjunto de características** (estado) dos objetos daquela classe
- A **definição é válida** para **todos os objetos** de uma classe
- As valores de todos os atributos de um objeto determinam o estado do mesmo



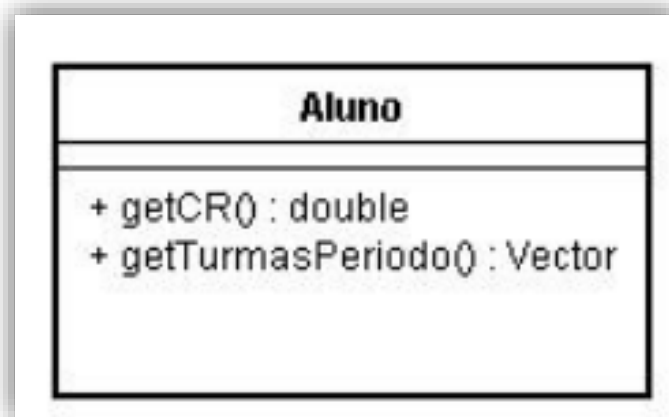
ELEMENTOS – DIAGRAMA DE CLASSES



ELEMENTOS — DIAGRAMA DE CLASSES

Métodos ou Operações

- É a **declaração de um serviço** que pode **ser solicitado** às **instâncias** de uma classe;
- A **declaração** de uma classe **é válida para todos** os **objetos** de uma classe
- Representam o **conjunto de operações** (comportamento) que a **classe fornece**



ELEMENTOS — DIAGRAMA DE CLASSES

Visibilidade

- Determina o nível de restrição de acesso (referência) a alguns elementos de modelagem da UML
- Aplicável às classes, as métodos/operações e aos atributos

ELEMENTOS — DIAGRAMA DE CLASSES

- Tipos de Visibilidades

- “+” público:

- Libera o acesso a ele de qualquer lugar que contenha a classe
 - Quando você define um método ou atributo como público, você está liberando o acesso a ele de qualquer lugar que contenha a classe.

- “#” protegido:

- Restringe acesso fora da classe, porém ele ainda é acessível de suas subclasses, ou seja, classes que herdaram os atributos dela.

ELEMENTOS — DIAGRAMA DE CLASSES

▪ Tipos de Visibilidades

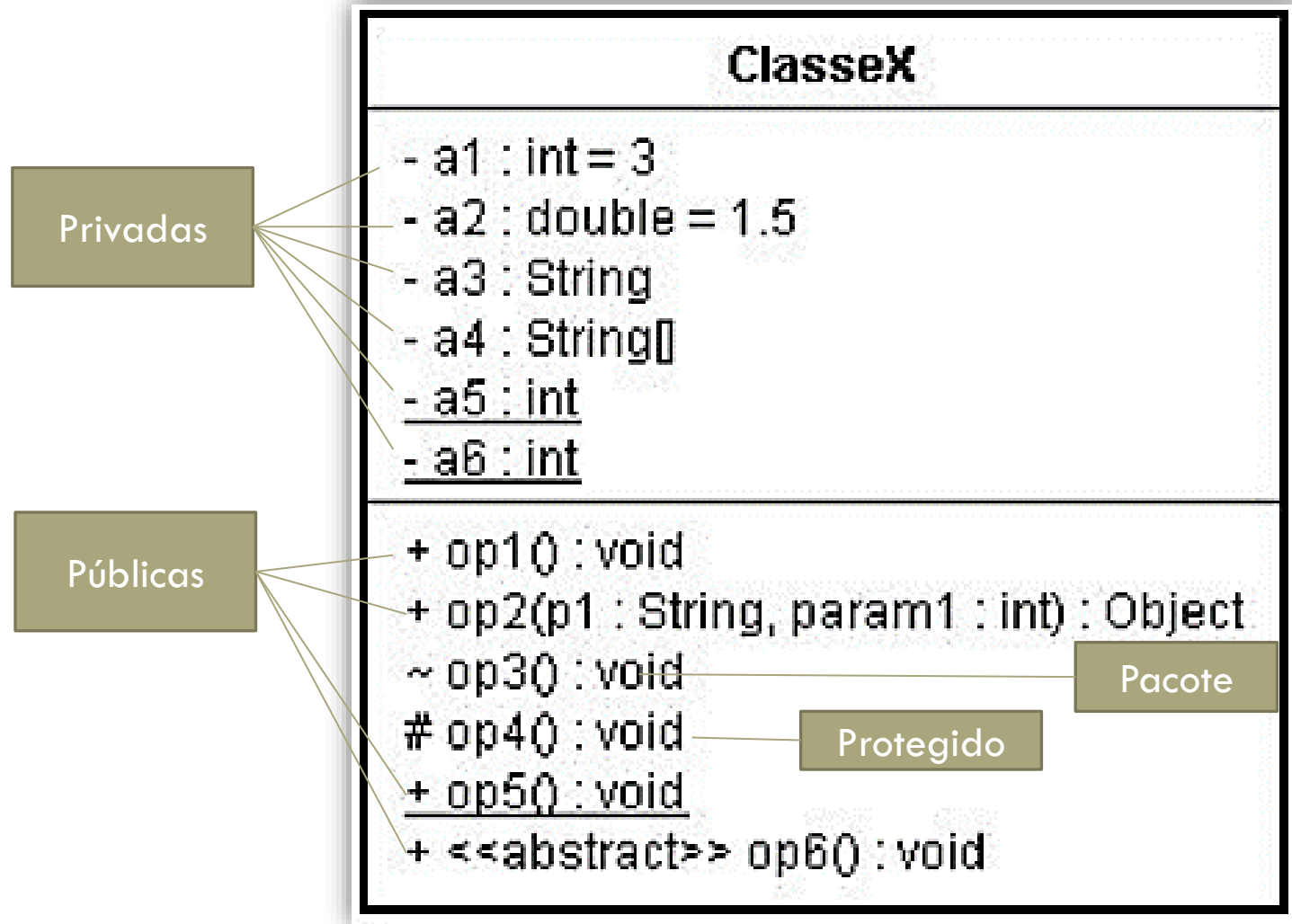
“_” privado:

- Visível somente na classe
- Esse modificador é mais restritivo, só da permissão de acesso a sua própria classe, em nenhum outro lugar é posso acessá-la.

“~” pacote:

- Pode ser referenciado por qualquer elemento de modelagem declarado no mesmo pacote.
- Para elementos externos, o elemento é tratado como privado

ELEMENTOS — DIAGRAMA DE CLASSES



ELEMENTOS — DIAGRAMA DE CLASSES

Elementos de um diagrama de classes

- Classes
- **Relacionamentos**
 - Associação
 - Agregação
 - Composição
 - Generalização
 - Dependência

RELACIONAMENTOS

- Um relacionamento é um termo geral que se aplica aos vários tipos de conexões possíveis entre os elementos de modelagem definidos na UML

RELACIONAMENTOS

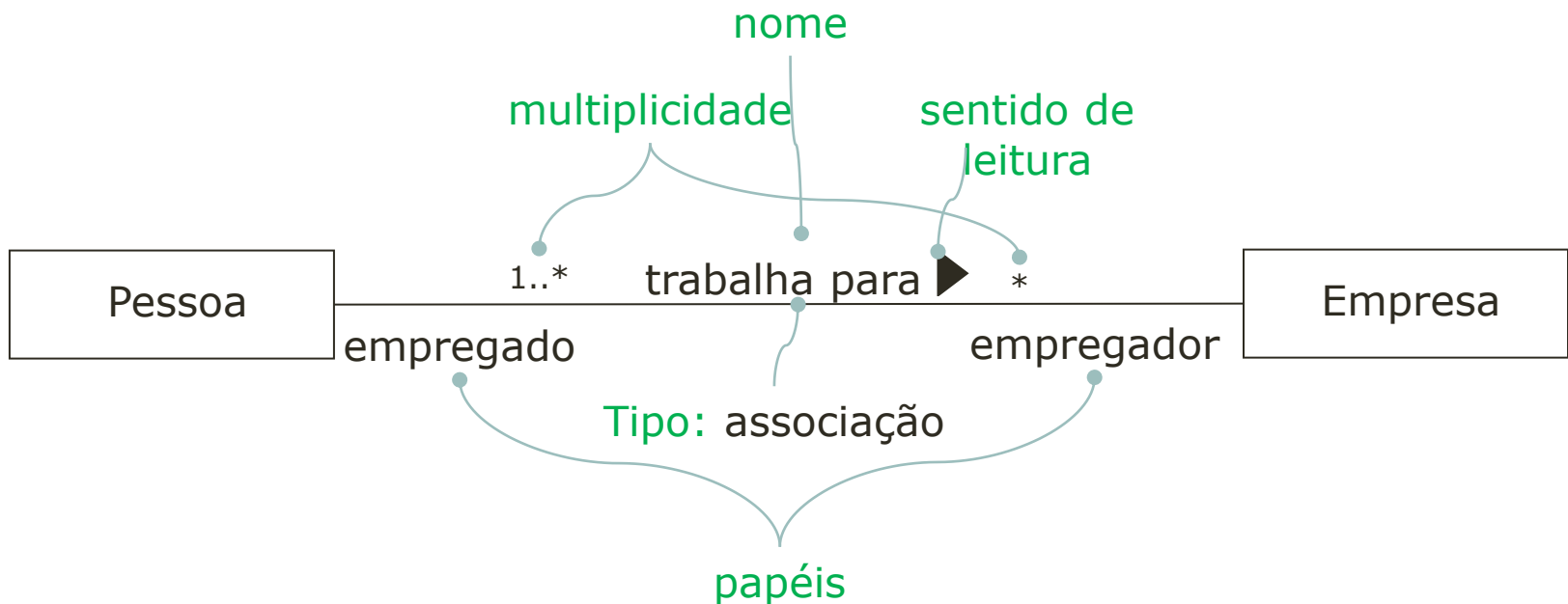
Relacionamentos

Os relacionamentos possuem:

- **Nome:** descrição dada ao relacionamento (faz, tem, possui,...)
- **Sentido de leitura**
- **Navegabilidade:** indicada por uma seta no fim do relacionamento
- **Multiplicidade:** 0..1, 0..*, 1, 1..*, 2, 3..7
- **Tipo:** **associação** (agregação, composição), **generalização** e **dependência**
- **Papéis:** desempenhados por classes em um relacionamento

RELACIONAMENTOS

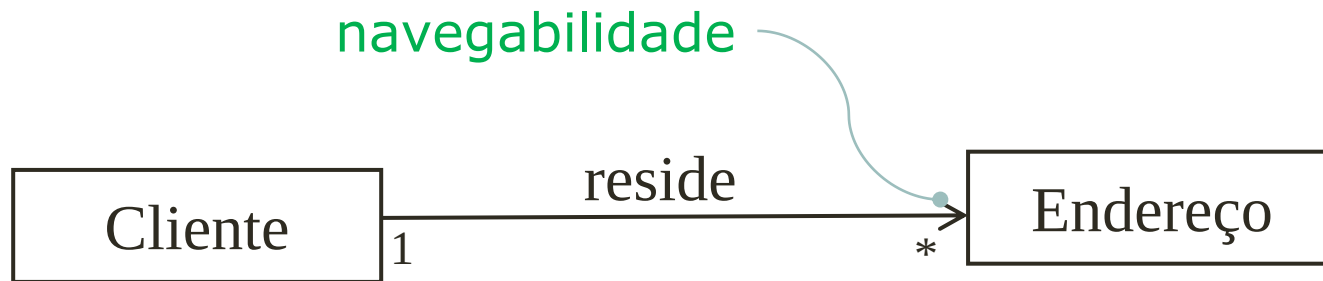
Relacionamentos



E a navegabilidade?

RELACIONAMENTOS

Relacionamentos



- O **cliente sabe quais são seus endereços**, mas o endereço não sabe a quais clientes pertence

RELACIONAMENTOS

Elementos de um diagrama de classes

- Classes
- **Relacionamentos**
 - Associação
 - Agregação
 - Composição
 - Generalização
 - Dependência

RELACIONAMENTOS: MULTIPLICIDADE

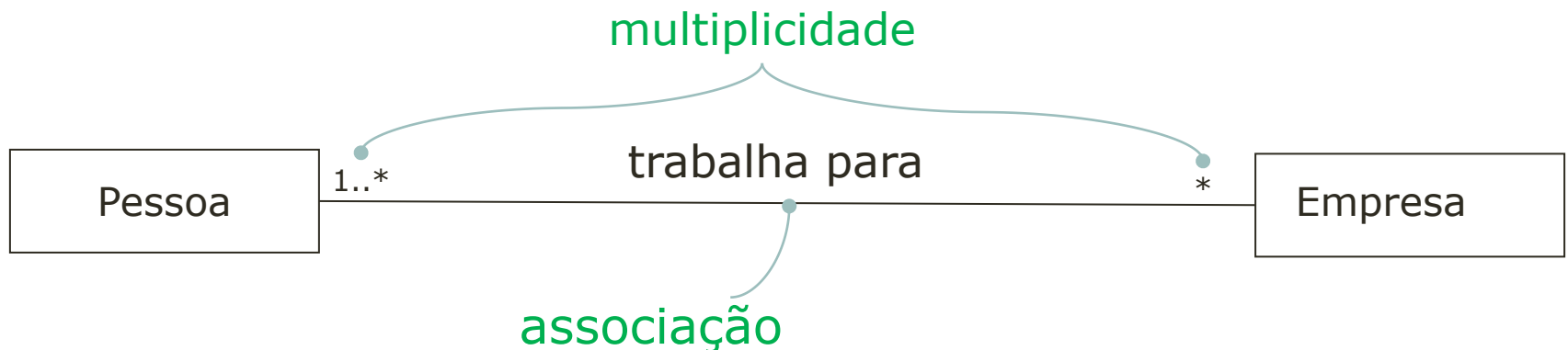
O conceito de multiplicidade é **muito importante** na **definição** das **regras que governam** as associações

A **multiplicidade** de **uma extremidade** de uma **associação** define o **número** de **objetos da classe** associada a **esta extremidade** que podem **estar conectados** a uma dada **instância da classe** associada à **extremidade oposta**.

RELACIONAMENTOS: ASSOCIAÇÃO

Indicadores de multiplicidade:

- 1 Exatamente um
- 1..* Um ou mais
- 0..* Zero ou mais (muitos)
- * Zero ou mais (muitos)
- 0..1 Zero ou um
- m..n Faixa de valores (por exemplo: 4..7) lê-se de quatro a sete
- 3,5,8 exatamente três, cinco ou oito

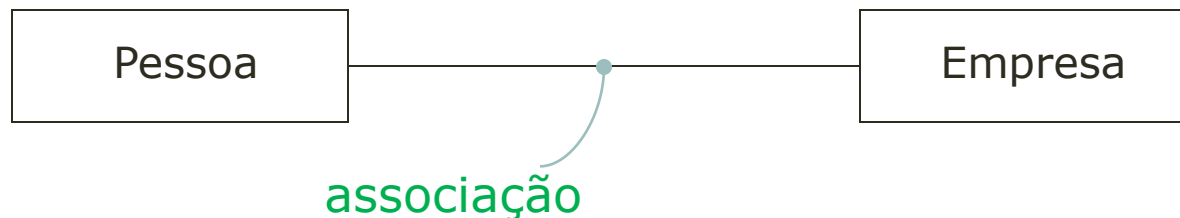


RELACIONAMENTOS: ASSOCIAÇÃO

Uma **associação** é um **relacionamento estrutural** que indica que os **objetos de uma classe estão vinculados a objetos de outra classe**.

Uma associação **é representada** por uma **linha sólida** conectando duas classes.

Uma **associação** deve ter um nome que **define a natureza** da associação



RELACIONAMENTOS: ASSOCIAÇÃO

Papel

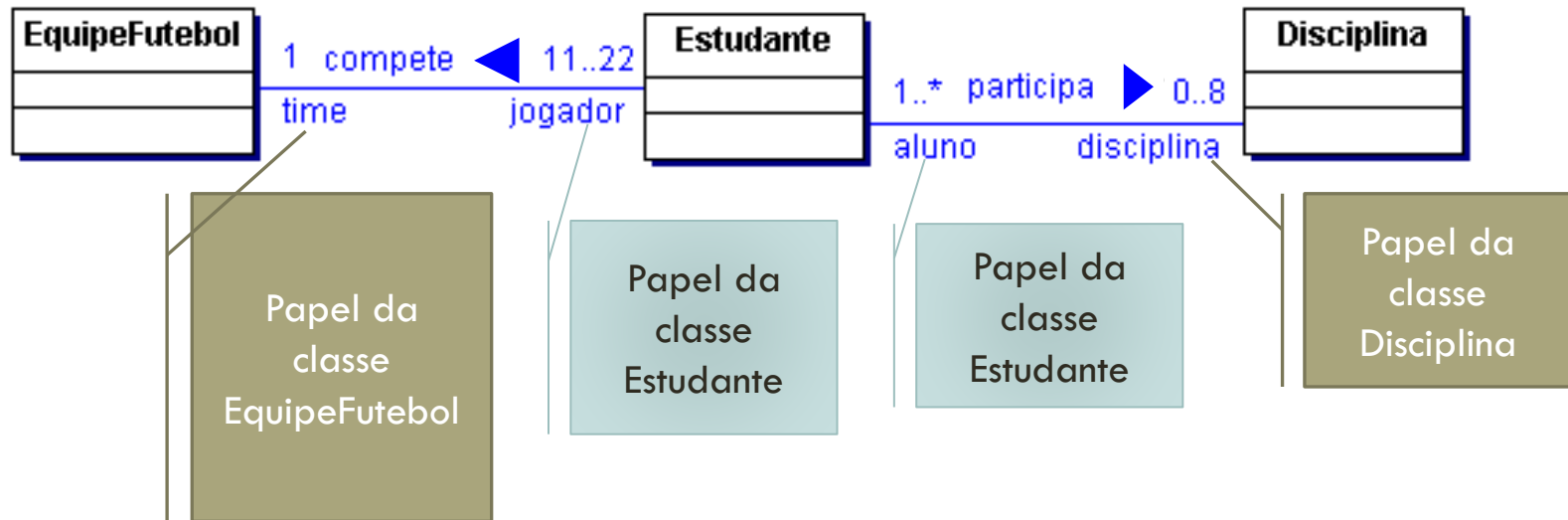
Quando uma **classe participa** de uma **associação**, ela **desempenha um papel** nesse relacionamento

O **papel** evidencia **a finalidade ou função** de **cada classe** de uma associação

Uma **mesma classe** pode **desempenhar** vários **papéis** em **diversas associações**

Na maioria das vezes, o **próprio nome da classe** define **o seu papel** na associação

RELACIONAMENTOS: ASSOCIAÇÃO



RELACIONAMENTOS: ASSOCIAÇÃO

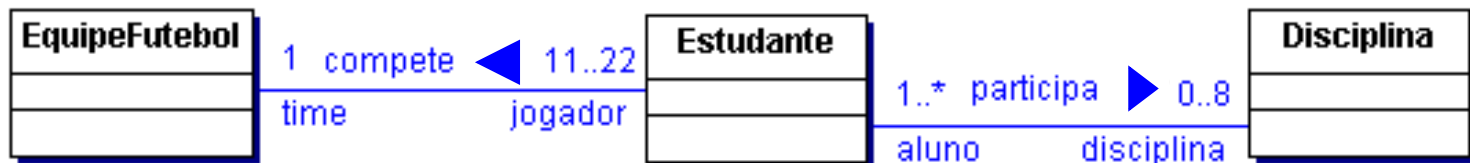
Exemplo:

Um **Estudante** pode ser
um **aluno** de uma Disciplina e
um **jogador** da Equipe de Futebol

Uma **equipe de futebol** tem entre **11 e 22** **estudantes** jogando

Cada **Disciplina** deve ser **cursada** por no **mínimo 1** **aluno**

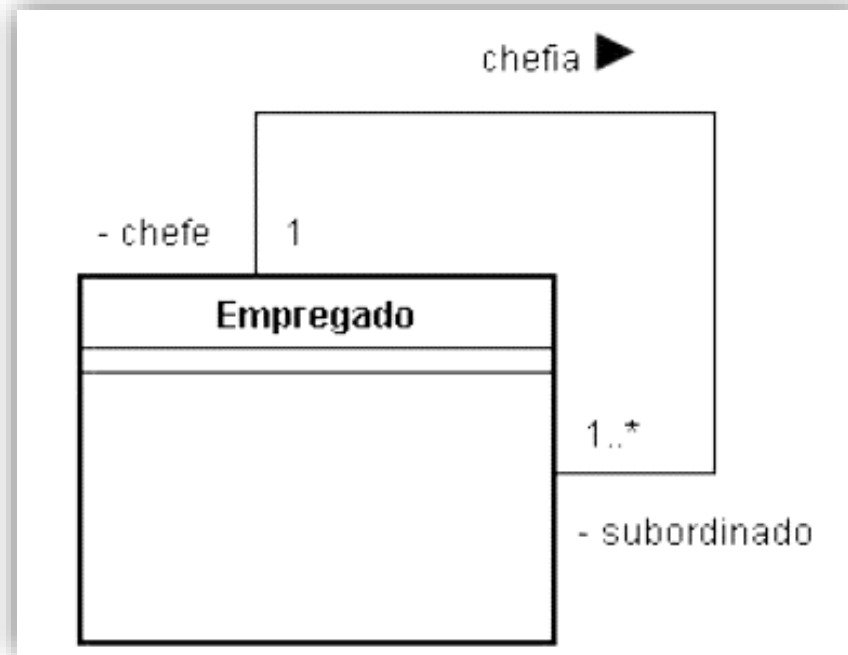
Um **aluno** pode cursar de **0 até 8** **disciplinas**



RELACIONAMENTOS: ASSOCIAÇÃO

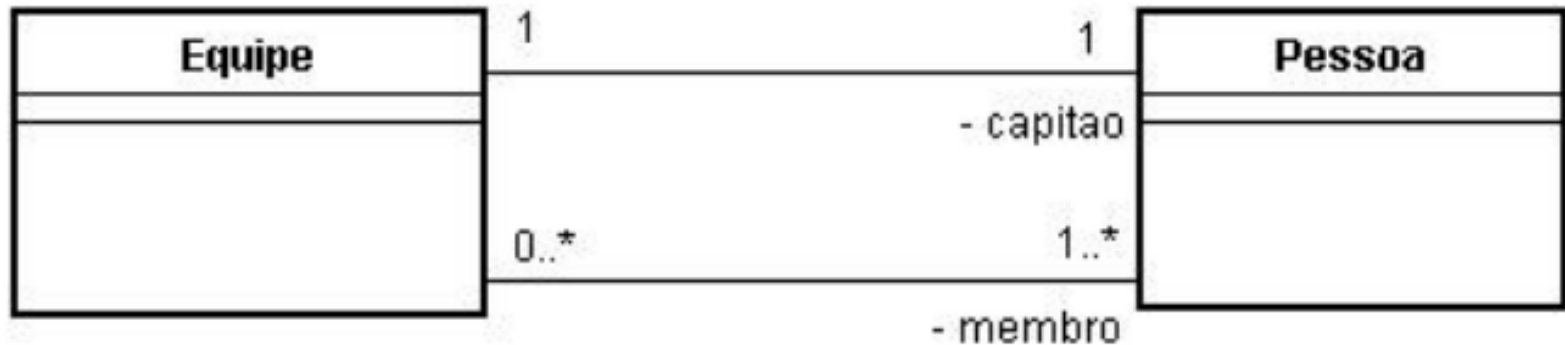
Papeis em alto relacionamentos

Em algumas situações, a **utilização dos papéis** é fundamental para o **completo entendimento** do modelo



RELACIONAMENTOS: ASSOCIAÇÃO

Outro uso de papel



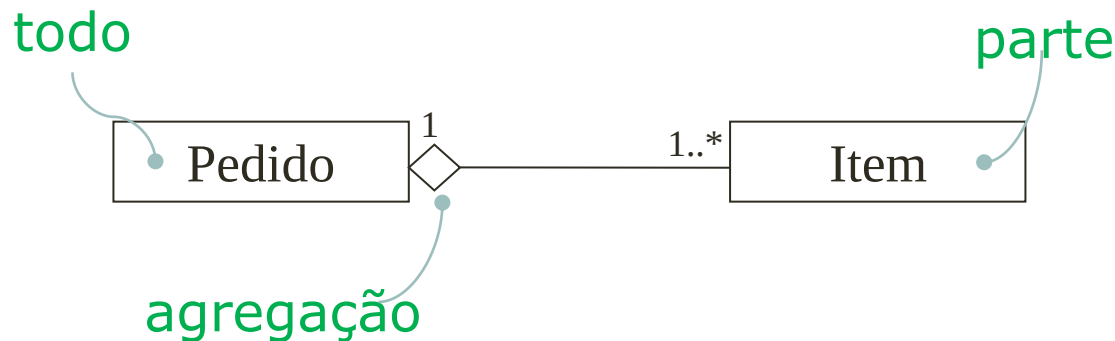
ELEMENTOS — DIAGRAMA DE CLASSES

Elementos de um diagrama de classes

- Classes
- **Relacionamentos**
 - Associação
 - **Agregação**
 - Composição
 - Generalização
 - Dependência

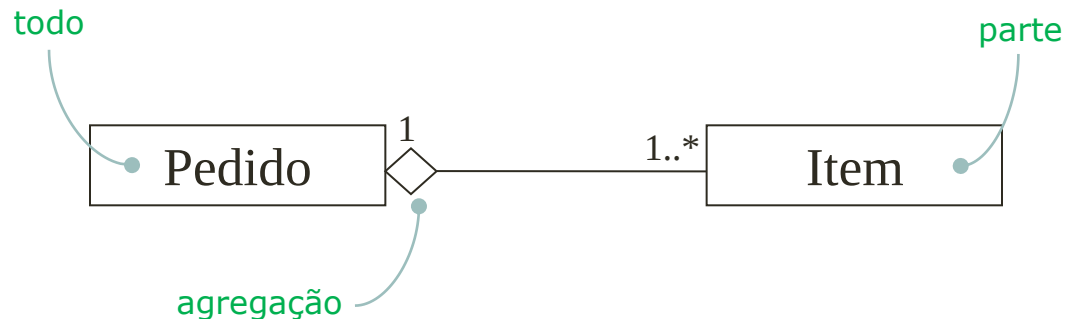
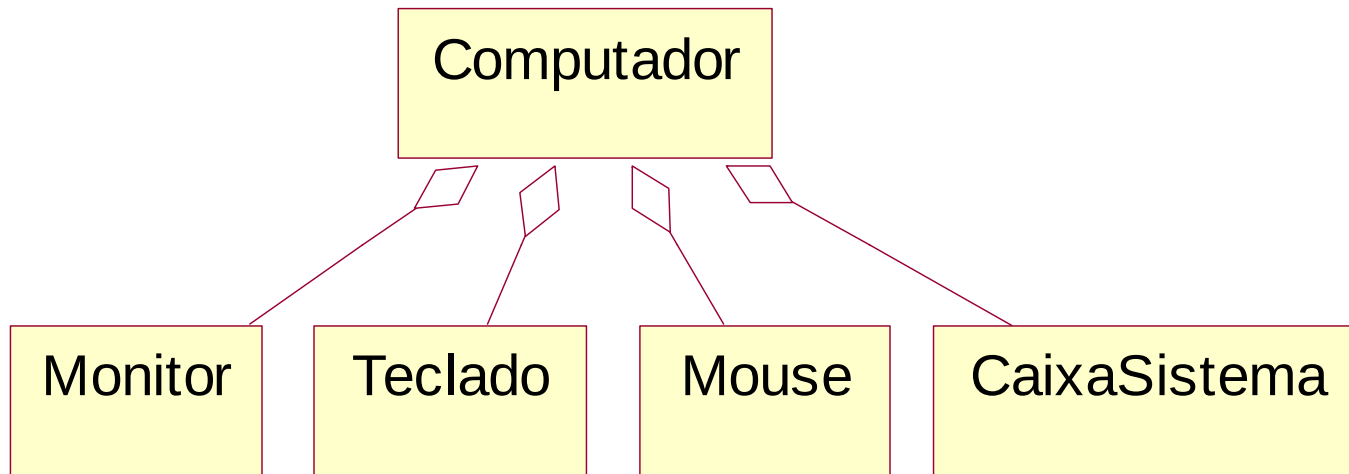
RELACIONAMENTOS: AGREGAÇÃO

- É um tipo especial de associação
- Utilizada para indicar “**todo-parte**” onde o todo é composto por partes.



- um objeto “**parte**” pode fazer parte de vários objetos “**todo**”

RELACIONAMENTOS: AGREGAÇÃO



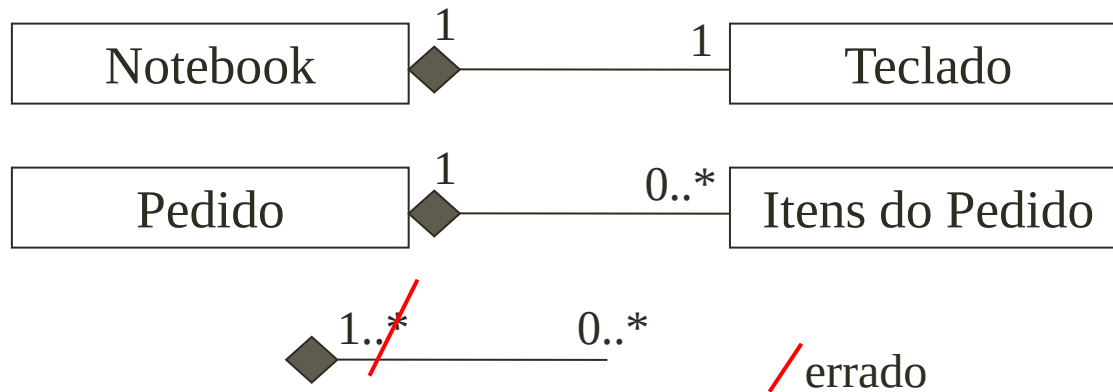
ELEMENTOS — DIAGRAMA DE CLASSES

Elementos de um diagrama de classes

- Classes
- **Relacionamentos**
 - Associação
 - Agregação
 - **Composição**
 - Generalização
 - Dependência

RELACIONAMENTOS: COMPOSIÇÃO

- É uma forma mais **“forte” da agregação**
- Os **objetos “parte”** só podem pertencer **a um único objeto “todo”** e têm o seu tempo de **vida coincidente com o dele**

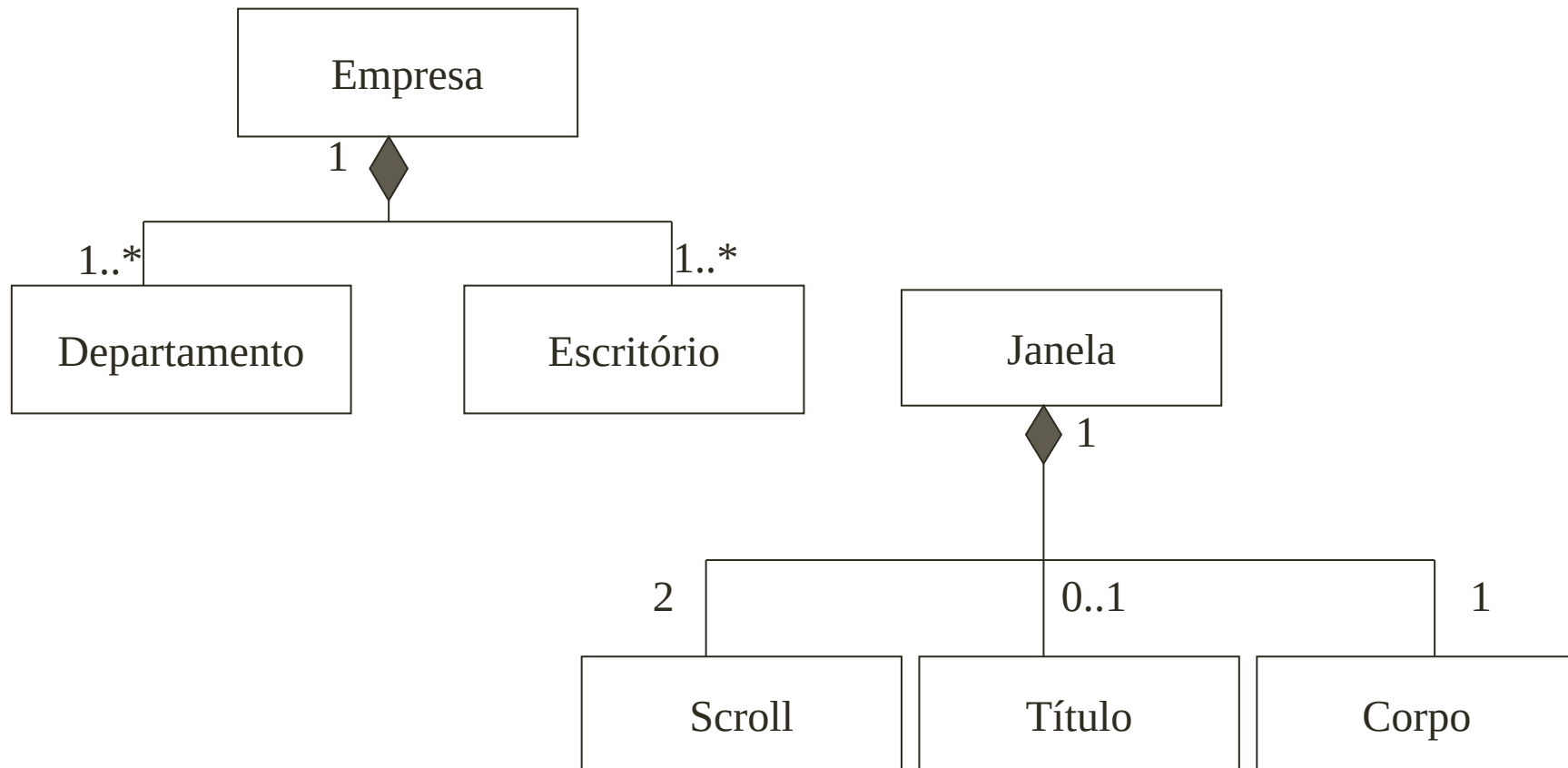


- Quando o **“todo” morre** todas as suas **“partes” também morrem**

RELACIONAMENTOS: COMPOSIÇÃO

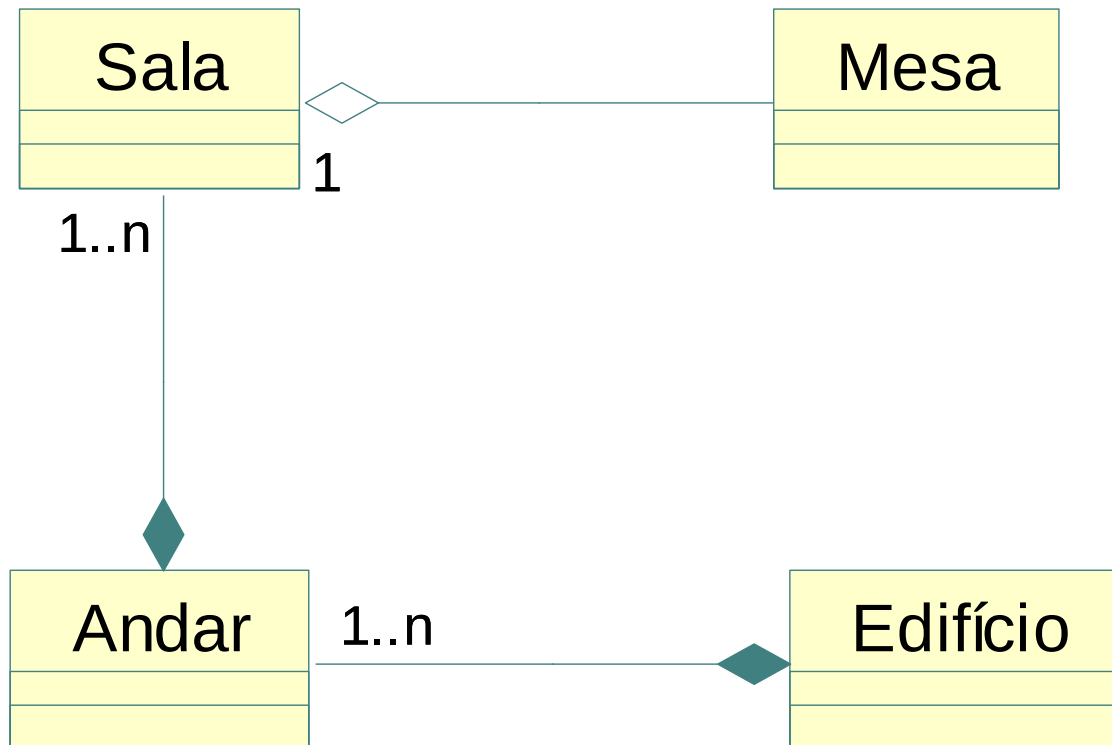
Relacionamento: Composição

Ex:



ELEMENTOS — DIAGRAMA DE CLASSES

Agregação **X** Composição



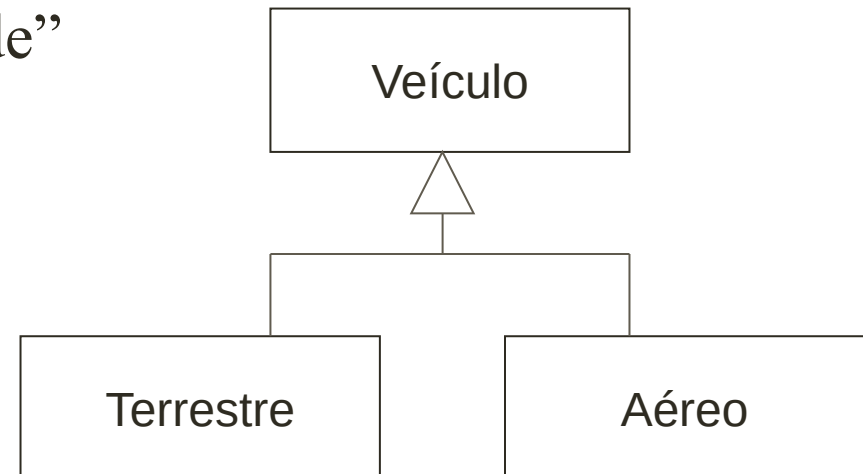
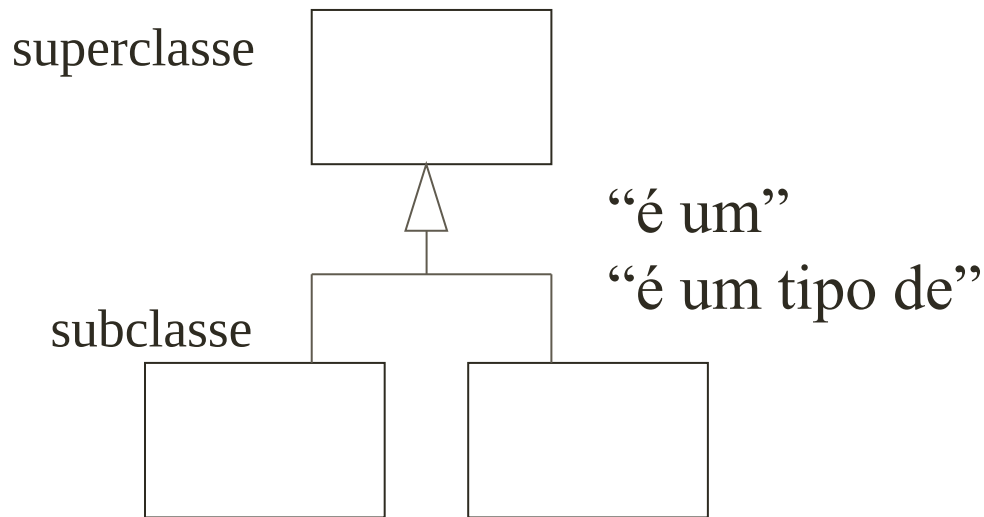
ELEMENTOS — DIAGRAMA DE CLASSES

Elementos de um diagrama de classes

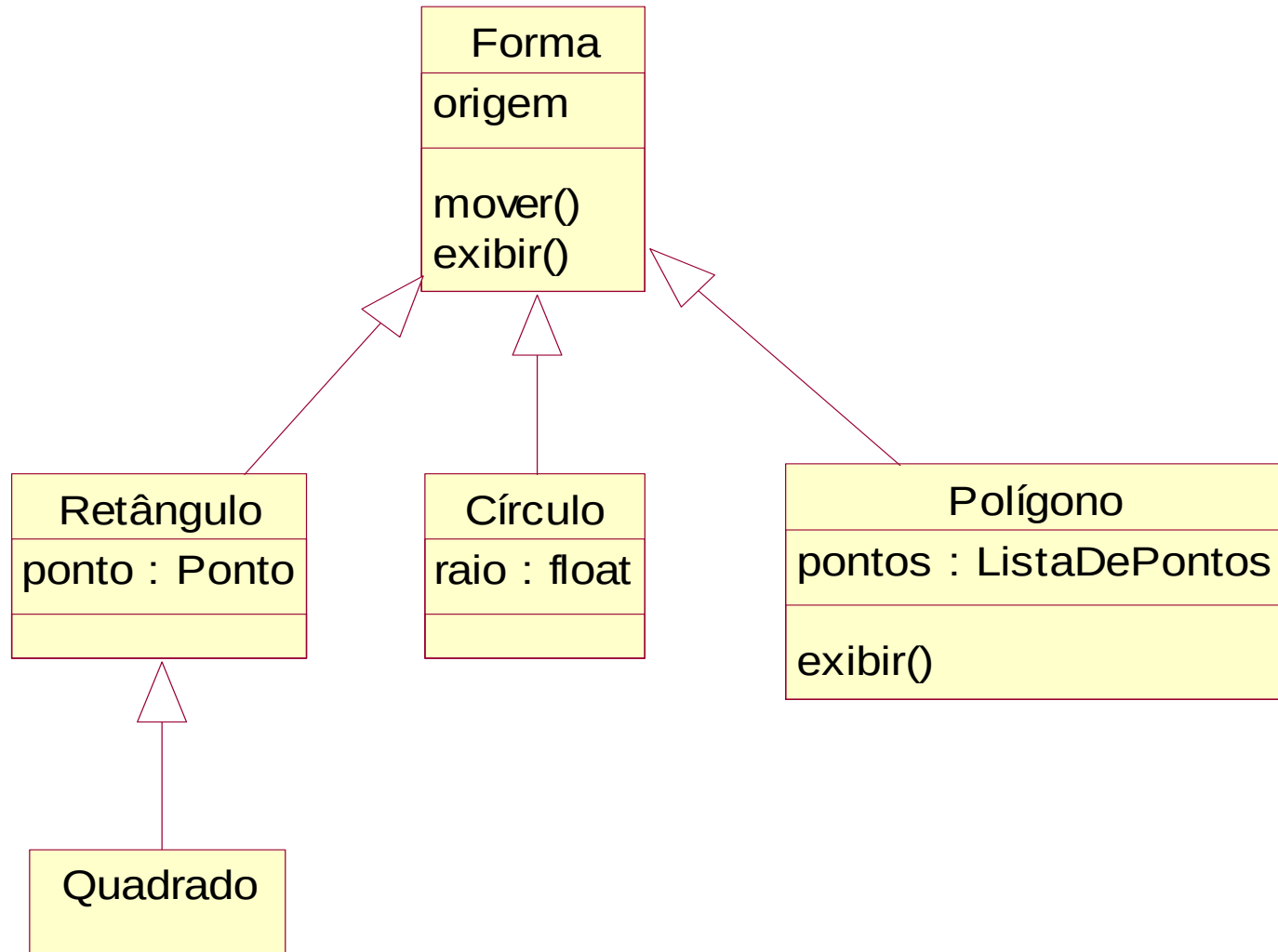
- Classes
- **Relacionamentos**
 - Associação
 - Agregação
 - Composição
 - **Generalização**
 - Dependência

RELACIONAMENTO: GENERALIZAÇÃO

- É um relacionamento **entre itens gerais (superclasses)** e **itens mais específicos (subclasses)**

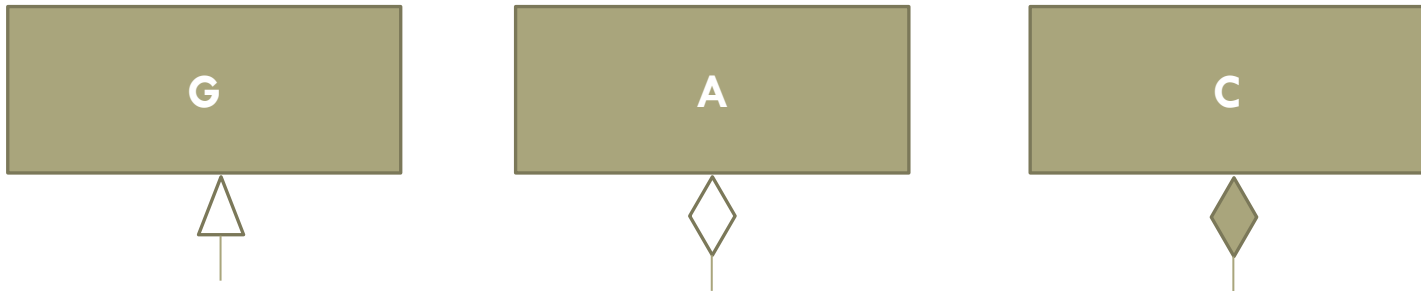


RELACIONAMENTO: GENERALIZAÇÃO



OBS:

Cuidado para não confundir:



Generalização = é um

Agregação = Tem-um, é-parte-de

Composição = Objeto parte só existe com o todo

ELEMENTOS — DIAGRAMA DE CLASSES

Elementos de um diagrama de classes

- Classes
- **Relacionamentos**
 - Associação
 - Agregação
 - Composição
 - Generalização
 - **Dependência**

RELACIONAMENTO: DEPENDÊNCIA

- Também conhecido como **classe associativa**
- Dependências são **relações de USO**.
- Uma Dependência *indica que mudanças em um elemento* (“o Servidor”) podem *afetar outro elemento* (“o Cliente”).

Ex:



RELACIONAMENTO: DEPENDÊNCIA

- Analise a associação abaixo, que relaciona os alunos às disciplinas por eles cursadas:

Ex:



A qual das classes **deve pertencer** o atributo **nota**, que **representa** a nota que **um aluno obteve** em uma disciplina

RELACIONAMENTO: DEPENDÊNCIA

- Analise a associação abaixo, que relaciona os alunos às disciplinas por eles cursadas:

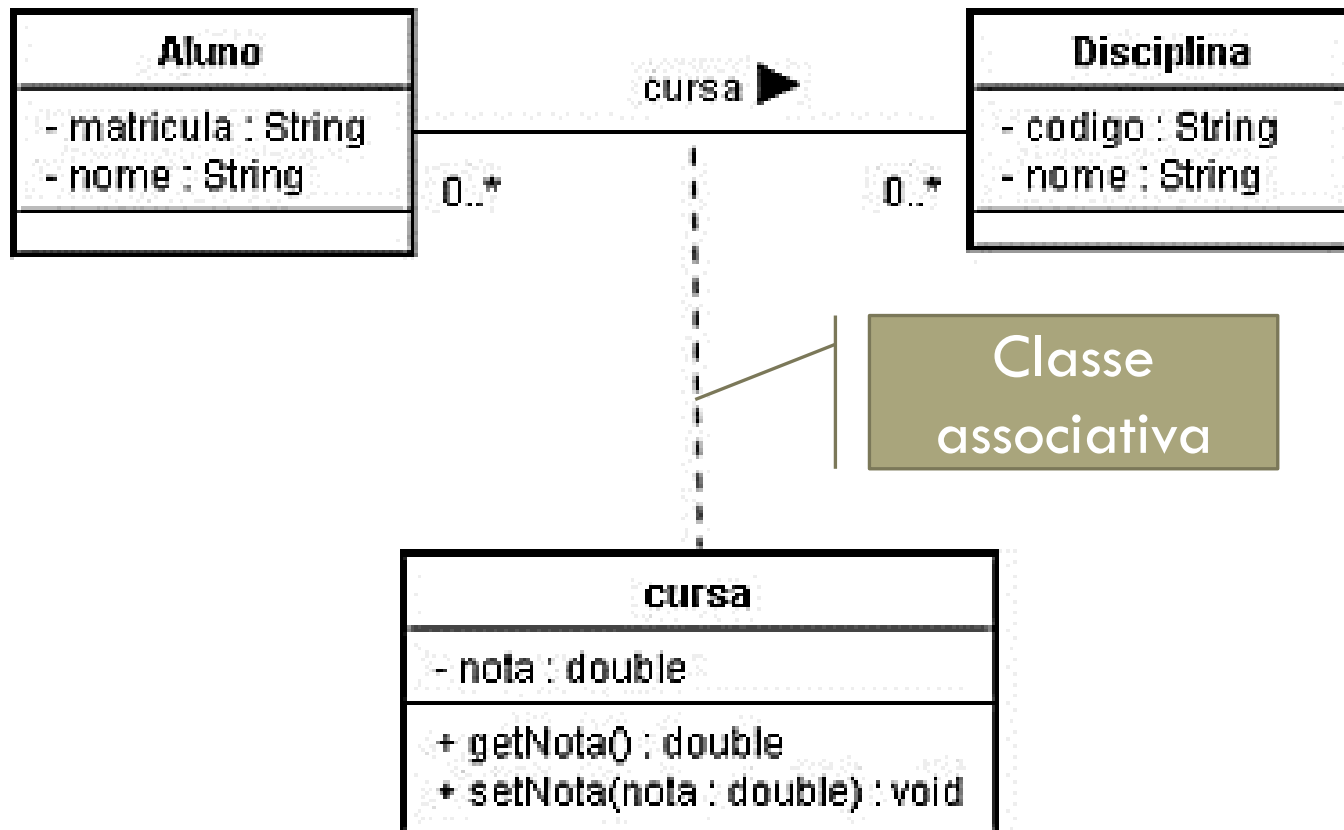
Ex:



A qual das classes **deve pertencer** o atributo **nota**, que **representa** a nota que **um aluno obteve** em uma disciplina

Resposta a nenhuma das classes, e sim à relação cursa

RELACIONAMENTO: DEPENDÊNCIA



REFERÊNCIAS

1. Sommerville, I.; **Engenharia de Software**. 10ed, Ed. Pearson, 2019. ISBN: 9788543024974.
2. Wazlawick, R.S., **Análise e Projeto de Sistemas de Informação Orientados a Objetos**, GEN LTC, 3ª edição, 2014.
3. Guedes, G. T. A. **UML 2** - Uma Abordagem Prática – 3ed. Novatec, 2018. ISBN: 9788575226469
4. Cockburn, A. **Escrevendo Casos de Uso Eficazes: Um Guia Prático para Desenvolvedores de Software**, Bookman, 2005.
5. Blaha, M.; Rumbaugh, J.; **Modelagem e Projetos Baseados em Objetos com UML 2**. 2ed, Ed. Campus, 2006. ISBN:8535217533.